

Calloway OS

Omni-Core Telemetry Station

Project Report

Author	Logan Calloway
Platform	XIAO ESP32-C6
Display	ILI9341 2.8" TFT (320x240)
Version	v1.6
Date	April 2026

“A centralized, glanceable engineering dashboard designed to monitor a workspace and provide real-time environmental and network data from a single, always-on embedded device.”

Abstract

This report documents the design, development, and architecture of the Omni-Core Telemetry Station, a custom desktop IoT dashboard built on the XIAO ESP32-C6 microcontroller running Calloway OS.

The goal of this project was straightforward: build a device that never crashes, always knows where it is, and fills a screen with real, useful data pulled from both the internet and local sensors. Most IoT projects work until they don't. A dropped WiFi connection, a bad API response, or a stalled HTTP request can bring everything down silently. This project was designed from the ground up to handle those situations gracefully and recover on its own.

The device pulls time from NTP servers, weather from OpenWeatherMap, and reads live environmental data from a TSL2591 light sensor and a BME280 temperature, humidity, and pressure sensor. Location and timezone are detected automatically via IP geolocation, which means the device works correctly whether it is sitting on a desk at home or plugged into a power outlet in a camper van. Everything updates itself with no configuration needed after first setup.

The software architecture enforces a strict separation between data fetching, display rendering, and system coordination. This makes the codebase easy to extend as new sensors arrive and keeps individual functions small and honest about what they do.

Keywords: ESP32-C6, IoT, embedded systems, telemetry, NTP, OpenWeatherMap, ILI9341, TSL2591, BME280, C++, Arduino framework

Contents

1	Introduction	3
2	Project Goals	3
3	Hardware Design	4
3.1	Platform	4
3.2	Display	4
3.3	Sensors	4
3.4	Backlight Control	4
3.5	Decoupling	5
4	Software Architecture	5
4.1	Three-Layer Design	5
4.2	Non-Blocking Loop	5
4.3	Canvas Buffering	6
4.4	Color Palette System	6
4.5	Settings Menu	6
4.6	Settings Persistence	6
5	Reliability Design	6
6	Development History	7
7	Current Status and Future Work	8
7.1	What’s Working Now	8
7.2	Next Steps	8
8	Conclusion	9

Introduction

Most IoT devices are built to work in one place, on one network, with one fixed configuration. They tend to be brittle in ways that only show up after you have been using them for a while. A router change breaks the WiFi, a timezone shift breaks the clock, or a brief power outage leaves the device in a state it can never recover from on its own.

The Omni-Core Telemetry Station was built to be different. The core question driving every design decision was: *what happens when something goes wrong, and how does the device get itself back to a good state without any help?*

That question shaped everything. WiFi credentials are stored in non-volatile memory and the device reconnects automatically if the signal drops. Location and timezone are detected from the network rather than hardcoded, so moving the device to a different city or plugging it in while traveling does not break anything. A hardware watchdog timer resets the chip if the main loop ever stalls. HTTP requests have timeouts so a slow API server cannot freeze the display. A 6-hour reconnect limit triggers a clean restart if WiFi never comes back.

The display side was designed with the same thinking. Off-screen canvas buffers are used for the clock, date, and local sensor readings, so every region of the screen updates atomically with no visible flicker. Only the parts of the display that actually changed get redrawn on each tick. The color palette is defined in a single place and everything on screen responds to a Night Mode toggle that shifts every element to a red palette, which protects the user's circadian rhythm during late-night sessions.

This report covers the hardware design, software architecture, key engineering decisions, and the development history of the project from initial prototype to the current version. The full source code is available at <https://github.com/logancalloway/calloway-os>.

Project Goals

The Omni-Core was built around a few core requirements that stayed constant throughout development.

- **Reliability above everything.** The device should run unattended for months without intervention. Every failure mode should have a recovery path.
- **Location independence.** The device should work correctly anywhere with an internet connection, whether at home, at a desk in another city, or in a vehicle. No manual reconfiguration required.
- **Real data only.** Nothing on the display should show a placeholder or a zero when real data is not available yet. The display waits for valid data before drawing anything.
- **Glanceability.** The dashboard should communicate everything the user needs at a glance, including time, date, weather, local environmental readings, and WiFi status, without requiring any interaction.

- **Expandability.** The architecture should make it easy to add new sensors without restructuring existing code.

Hardware Design

Platform

The XIAO ESP32-C6 was chosen as the platform for several reasons. It runs at 160MHz with 512KB of RAM and 4MB of flash, which is significantly more headroom than the project currently needs and leaves room for future expansion. The built-in WiFi and Bluetooth stack handles all network communication without any additional hardware. The RISC-V core architecture is increasingly relevant in professional embedded work, making this a good platform to develop skills on.

Display

The ILI9341 2.8-inch TFT display provides a 320x240 pixel color screen over SPI. The display is driven by the Adafruit ILI9341 and GFX libraries. Three GFXcanvas16 off-screen buffers are used for the clock, date, and local sensor regions to prevent visible flickering during updates. Each region draws completely into RAM before a single SPI transfer pushes the finished frame to the screen.

Sensors

Sensor	Purpose	Status
TSL2591	Ambient light (lux)	Active
BME280	Temperature, humidity, pressure	Active

Both sensors communicate over I2C on pins D4 (SDA) and D5 (SCL) at the default 100kHz clock speed. The TSL2591 has a dynamic range of up to 88,000 lux, making it suitable for both dark room and bright daylight readings. Its lux output drives a vertical bar on the display and will eventually control the backlight brightness automatically via a PFET transistor once the PCB is assembled. The BME280 provides local temperature, relative humidity, and barometric pressure, which are displayed in the top left of the dashboard and update every 2 seconds.

Backlight Control

The backlight is currently driven directly from 3.3V with no active control. The final PCB design uses an AO3401 P-channel MOSFET driven from 5V, with the gate controlled by a PWM signal from pin D2. This inverts the PWM logic, meaning a high PWM value dims the display rather than brightening it, which is already accounted for in the software. The TSL2591 lux reading will map directly to the PWM duty cycle for automatic ambient

brightness control. The menu already includes Brightness and Auto Dim settings that are fully implemented and simply waiting on the hardware connection.

Decoupling

The current breadboard prototype has experienced occasional brownout resets during WiFi association due to the current spike from the radio. The PCB design includes 100nF and 10µF decoupling capacitors on both the 3.3V and 5V rails, placed close to the IC power pins, which should eliminate this issue.

Software Architecture

Three-Layer Design

The codebase is organized into three distinct layers that never cross into each other's responsibilities.

Layer	Responsibility
Data Layer	Fetches data from the network or hardware sensors and writes results to shared global variables prefixed with <code>g_</code> . Never touches the display.
Display Layer	Reads <code>g_</code> globals and draws to the screen. Never makes network calls or reads hardware directly.
Coordinator	The main <code>loop()</code> function. Owns all timing decisions, gates network calls behind WiFi status checks, and decides when each layer runs.

This separation means that adding a new sensor is just filling in two functions. One reads the hardware and writes to a global, and the other reads that global and draws to the screen. Nothing else in the codebase needs to change.

Non-Blocking Loop

The main loop is entirely non-blocking. Every repeating task is driven by a `millis()` interval check rather than a `delay()` call. This keeps the loop running continuously so the button always feels instant and no task can block another.

```
1 // All timing decisions live in loop() -- nothing blocks
2 if (currentMillis - lastClockTime >= clockInterval) {
3     updateTimeDisplay();
4     lastClockTime = currentMillis;
5 }
```

Canvas Buffering

Three off-screen canvas buffers prevent flickering on the regions of the display that update most frequently. Without buffering, each drawing operation is a separate SPI transaction and the screen passes through partially drawn states that the eye sees as a flash. With a canvas, all drawing operations happen in RAM first and the finished frame is pushed to the display in a single atomic SPI transfer.

Canvas	Size	Update Rate
clockCanvas	260x45 px	Every second
localCanvas	150x55 px	Every 2 seconds
dateCanvas	240x20 px	Every minute

Color Palette System

All display colors are defined in a single `ColorPalette` struct returned by `getColorPalette()`. When Night Mode is active, every element on screen, including the clock, date, weather, WiFi icon, and lux bar, shifts to a red palette without any per-function color logic. Adding a new display element just requires reading from the palette rather than hardcoding colors.

Settings Menu

A single hardware button provides access to a scrollable settings menu using short press to navigate and long press to select. The button was deliberately kept as the primary input method rather than using a touch screen. It is simpler, more reliable, requires no calibration, and works the same way every time. Current settings include Night Mode, temperature unit (F/C), military time (12/24hr), brightness control, auto dim, and WiFi reset. The menu is designed to scale and adding a new setting requires one new global, one new string in the items array, and one new case in the switch statement. The scrollbar updates automatically.

Settings Persistence

All user settings are saved to non-volatile storage immediately when changed and restored on every boot before the first frame draws. This means the device always starts up exactly as the user left it. Each setting is stored under a dedicated key in the `"settings"` NVS namespace, which is kept completely separate from the `"wifi-gate"` namespace used for WiFi credentials. A deep reset clears both namespaces simultaneously.

Reliability Design

Reliability was treated as a first-class requirement, not an afterthought. The following mechanisms work together to ensure the device recovers from any failure without manual intervention.

Mechanism	What it handles
Hardware watchdog (30s)	Resets the chip if <code>loop()</code> ever stalls, catching HTTP hangs, driver issues, or any other freeze
HTTP timeouts (5s response, 3s connect)	Prevents a slow or unresponsive API server from freezing the display
WiFi auto-reconnect (60s retry)	Quietly retries the connection in the background without disrupting the display
NTP resync debounce	Only resyncs time if it has been more than 60 seconds since the last sync, preventing hammering during a spotty connection
6-hour restart	If WiFi never comes back after 6 hours, the device restarts cleanly rather than sitting in a degraded state
Daylight saving detection	Resyncs location and time at 2:01 AM once per year to handle DST transitions automatically

The watchdog is intentionally armed after all blocking startup operations complete. This prevents false triggers during the WiFi connection loop on boot, which can take an unpredictable amount of time depending on network conditions.

Development History

The project was developed iteratively, with each version focused on a single clear goal.

v1.0 – Core Functionality

Established the foundation of Calloway OS. Implemented NTP-synchronized timekeeping and weather data from OpenWeatherMap. This version proved that the core concept, a glanceable dashboard driven entirely by internet data, was viable on the ESP32-C6 platform.

v1.1 – Network Intelligence

Replaced hardcoded WiFi credentials with a WiFiManager captive portal for first-time setup. Added automatic location detection via ip-api.com, which allowed timezone and weather city to be set dynamically from the network. The device became truly self-configuring. Plug it in anywhere and it figures out where it is.

v1.2 – Architecture Overhaul

Restructured the entire codebase around the three-layer architecture. Introduced non-blocking `millis()` intervals, separated data and display functions, and moved all timing decisions into `loop()`. This version transformed working code into well-engineered code.

v1.3 – Reliability

Added the hardware watchdog, HTTP timeouts, WiFi health monitoring with auto-reconnect, the deep reset system, and the 6-hour restart fallback. This version was focused entirely on making the device reliable enough to run unattended for months.

v1.4 – Hardware and Display

Brought the TSL2591 light sensor online. Added the ILI9341 TFT display with canvas-buffered rendering to prevent flickering. Implemented the Night Mode color palette system and the vertical lux bar on the display.

v1.5 – Settings and User Control

Added a button-driven scrollable settings menu with Night Mode, Celsius/Fahrenheit toggle, Military Time (24hr clock), Brightness control, Auto Dim (stubbed for when the PFET arrives), and WiFi reset with confirmation. Implemented short/long press detection and a scrollbar indicator that scales automatically as new items are added. All settings persist across reboots via NVS, so the device boots back up exactly as the user left it. The brightness adjust mode uses a distinct yellow highlight so the user always knows when they are in adjustment mode versus navigation mode.

v1.6 – Local Sensor Integration

Brought the BME280 temperature, humidity, and pressure sensor online. Local environmental readings now display in the top left of the dashboard and update every 2 seconds. Added canvas buffering for the local sensor area and the date line, eliminating the last remaining sources of display flicker. The I2C bus was also investigated during this version when a counterfeit SGP30 air quality sensor was identified by its blank serial ID memory and hardcoded impossible output values. The fake module was removed from the project and a genuine replacement has been ordered.

Current Status and Future Work

What's Working Now

The device currently displays a live NTP-synchronized clock with 12 or 24-hour format, the current date, internet weather data including temperature, daily high and low, and sky conditions, local temperature, humidity, and barometric pressure from the BME280, a live ambient light bar from the TSL2591, a WiFi signal strength indicator, and a full settings menu accessible via the hardware button. All screen regions update without flickering thanks to canvas buffering throughout.

Next Steps

- **SGP30 air quality.** A genuine Adafruit SGP30 has been ordered to replace the counterfeit module. Once it arrives, live VOC and eCO2 readings will be added to

the bottom of the dashboard with NVS baseline persistence across reboots.

- **PFET backlight control.** Wiring the AO3401 MOSFET will activate the already-implemented brightness and auto dim system. Two commented lines in the code are all that need to be uncommented once the hardware connection is made.
- **PCB fabrication.** Moving from the breadboard prototype to a custom PCB with proper decoupling, clean routing, and all components soldered permanently.
- **Sleep Timeout.** An additional menu setting to dim or turn off the display after a configurable period of inactivity.

Conclusion

The Omni-Core Telemetry Station started as a desk clock and grew into something more interesting. It is a small device that knows where it is, keeps itself alive, and shows useful real-world data without being told what to do. The engineering philosophy that shaped it, thinking about failure modes before writing code, keeping functions honest about what they do, and making the architecture easy to extend, is the same philosophy that shows up in professional embedded systems at every scale.

The hardware is nearly complete. Both environmental sensors are online and working, the display renders cleanly with no flicker, and all user settings survive reboots. What remains is finishing the backlight circuit, moving to a proper PCB, and adding the air quality sensor once the genuine module arrives. The software is ready for all of it.